DAT22-1

Scraping, Pandas II & Algorithmic Thinking

Advanced pandas, web scraping, and algorithmic reasoning on real data

---

Collect your own data, then process it efficiently.

Albert School — 28 hours

Contents

1 Advanced pandas

This course assumes pandas I: loading, inspecting, selecting (`loc`, `iloc`, boolean indexing), missing-value handling, `groupby`, merging, reshaping, and `apply`. It extends those operations.

1.1 Multi-level indexing

A DataFrame index (rows or columns) can hold more than one level, forming a `MultiIndex`. This represents data keyed by a combination of fields.

```
import pandas as pd

df = df.set_index(["country", "year"])   # two-level row index
df.loc[("France", 2020)]                 # select one combination
df.loc["France"]                         # select all rows under outer level
df.xs(2020, level="year")                # cross-section on an inner level
```

`groupby` on several keys produces a multi-level result. `unstack` moves an index level into the columns; `stack` does the reverse.

1.2 Window functions

A window function computes an aggregate over a sliding or growing set of rows rather than the whole column.

> **Definition 1** A **rolling** window of size k computes an aggregate over the last k rows for each position. An **expanding** window computes an aggregate over all rows from the start up to each position.

```
df["roll"] = df["value"].rolling(window=3).mean()   # last 3 rows
df["cum"]  = df["value"].expanding().sum()           # all rows so far
```

The first $k-1$ rows of a rolling window are `NaN` because the window is not yet full.

1.3 Time-indexed DataFrames

When the index is a `DatetimeIndex`, pandas supports time-based operations.

```
df["date"] = pd.to_datetime(df["date"])
df = df.set_index("date").sort_index()
df.loc["2020-01"]                # partial-string slice: all of Jan 2020
```

```
df.resample("M").sum()            # aggregate into monthly buckets
df["value"].rolling("7D").mean()  # time-based rolling window
```

`resample` changes the sampling frequency; rolling windows on a time index may be specified as a duration (e.g. `"7D"`) rather than a row count.

2 Algorithmic complexity

Definition 2 **Big O notation** describes how the number of operations an algorithm performs grows as the input size n grows, keeping only the dominant term. $O(n)$ means the work grows in proportion to n; $O(n^2)$ means it grows in proportion to n^2.

A loop over n rows is $O(n)$. A loop nested inside another loop over the same data is $O(n^2)$: for $n = 10\{,\}000$ rows, that is 10^8 operations.

A vectorized pandas operation applies the computation across the whole column in one call (executed in compiled code), and is $O(n)$ in the number of elements. Replacing a nested Python loop over a DataFrame with a vectorized operation therefore changes $O(n^2)$ work into $O(n)$ work, which is why it matters at scale: as n grows, the gap between the two widens without bound.

3 Performance profiling

Profiling means measuring where execution time is spent, so the slowest part (the bottleneck) can be identified and replaced.

```
import time
start = time.perf_counter()
result = do_work(data)
print(time.perf_counter() - start, "seconds")
```

In a notebook, `%timeit expr` measures the average run time of an expression, and `%prun call()` reports time per function call.

The procedure is: measure the pipeline, find the step taking the most time, replace it with a more efficient approach (typically a vectorized operation in place of a Python loop), and measure again to confirm the improvement.

4 Recursion

Definition 3 A **recursive function** is a function that calls itself on a smaller part of the problem. It requires a **base case** — an input handled directly without recursing — so the recursion terminates. Each call is placed on the **call stack** and removed when it returns; without a base case the stack grows until the program fails.

Recursion fits data that is itself nested. Parsing nested JSON:

```
def walk(node):
    if isinstance(node, dict):          # recurse into each value
        for value in node.values():
            walk(value)
    elif isinstance(node, list):        # recurse into each element
        for item in node:
            walk(item)
```

```
    else:
        print(node)                      # base case: a leaf value
```

Walking an HTML tree follows the same shape: handle the current element, then recurse on its children, stopping at elements with no children.

5 Web scraping with requests and BeautifulSoup

Scraping means retrieving a web page and extracting data from its HTML.

```
import requests
from bs4 import BeautifulSoup

response = requests.get("https://example.com/page")
soup = BeautifulSoup(response.text, "html.parser")

title = soup.find("h1").text                 # first <h1>
items = soup.find_all("div", class_="item")  # all matching elements
for item in items:
    name = item.find("span", class_="name").text
    link = item.find("a")["href"]
```

`requests.get` fetches the page; `BeautifulSoup` parses the HTML into a tree. `find` returns the first matching element, `find_all` returns all of them; element text is read with `.text` and attributes with `["attr"]`.

6 Handling scraping challenges

- **Pagination**: data spread across multiple pages. Loop over page URLs (or follow the "next" link) until no further page exists.
- **Rate limiting**: a server may reject requests sent too quickly. Pause between requests with `time.sleep`.
- **Missing elements**: `find` returns `None` when nothing matches; accessing `.text` on `None` raises an error. Check for `None` before using the result.
- **Nested structures**: navigate the parsed tree (e.g. `find` inside a `find_all` result) to reach data nested several elements deep.

```
import time
for page in range(1, n_pages + 1):
    response = requests.get(f"https://example.com/list?page={page}")
    soup = BeautifulSoup(response.text, "html.parser")
    cell = soup.find("td", class_="price")
    price = cell.text if cell is not None else None
    time.sleep(1)        # respect rate limits
```

7 Storing scraped data

Scraped records are written to a structured format.

```
import pandas as pd
df = pd.DataFrame(records)
df.to_csv("data.csv", index=False)      # CSV

import sqlite3
conn = sqlite3.connect("data.db")
df.to_sql("items", conn, if_exists="replace", index=False)   # SQLite
```

CSV is a plain-text table; SQLite is a single-file relational database queryable with SQL. The collection process must be documented: the source URL, the date collected, and the meaning of each field.

8 Legal and ethical boundaries

Scraping is not always permissible. Before scraping, check:

- the site's **Terms of Service**, which may prohibit automated collection;
- the site's `robots.txt`, which states which paths automated clients may access;
- whether the data is **personal data**, which carries legal restrictions;
- the load placed on the server — requests must be rate-limited so as not to disrupt the site.

Scraping is permissible when it respects these boundaries; when it does not, it must not be carried out.

9 Building a reusable data collection module

A collection module packages scraping logic into reusable functions with clear inputs, outputs, and error handling, rather than a one-off script.

```
def collect(url, n_pages=1):
    """Return a list of record dicts scraped from `url`."""
    records = []
    for page in range(1, n_pages + 1):
        try:
            response = requests.get(f"{url}?page={page}", timeout=10)
            response.raise_for_status()
        except requests.RequestException as error:
            print(f"skipping page {page}: {error}")
            continue
        records.extend(parse(response.text))
        time.sleep(1)
    return records
```

The inputs (URL, number of pages) and output (a list of records) are explicit, and network failures are caught with `try/except` so one failed page does not abort the whole collection.

10 Exploratory analysis on self-collected data

Once collected, the dataset is explored with the pandas operations above: inspect it, clean it, compute summaries with `groupby` and window functions, and examine relationships between columns.

The goal is to formulate at least three **data-driven hypotheses** — statements about the data that are suggested by what was observed and that could be checked with further analysis (for example, a difference between groups, or a trend over time visible in the collected records).

Exercises

1. Build a `MultiIndex` DataFrame from two key columns. Select one combination, all rows under the outer level, and a cross-section on the inner level.
2. Add a 3-row rolling mean and an expanding sum to a column, and explain why the first rows of the rolling result are `NaN`.
3. Convert a date column to a `DatetimeIndex`, slice one month by partial string, and resample the data to a monthly frequency.

4. Write a nested Python loop over a DataFrame and the vectorized equivalent. State the Big O of each and explain why the difference matters as n grows.
5. Profile a small pipeline, identify the slowest step, replace it with a vectorized operation, and measure the improvement.
6. Implement a recursive function that processes a nested JSON structure. Identify its base case and explain the role of the call stack.
7. Extract a list of records from a public web page using requests and BeautifulSoup.
8. Scrape a multi-page site, handling pagination, rate limiting, and missing elements without crashing.
9. Store a scraped dataset as both CSV and SQLite, and document the source, date, and fields.
10. For a chosen website, assess against its Terms of Service and `robots.txt` whether scraping it is permissible.
11. Refactor a scraping script into a reusable collection module with clear inputs, outputs, and error handling.
12. Conduct an exploratory analysis on a self-collected dataset and formulate at least three data-driven hypotheses.